



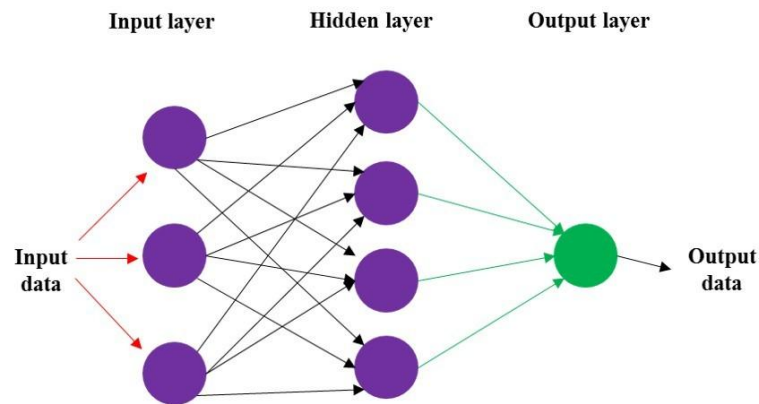
IITMO

Efficient Deep Learning — Quantization, Pruning & Distillation

Современные архитектуры нейронных сетей

Нейросети раньше

- Архитектура обычно представляла собой несколько последовательно идущих линейных (полносвязных) слоёв.
- Глубина и ширина таких сетей были относительно небольшими, поэтому общее число параметров оставалось умеренным.
- Типичный размер весов нейронной сети редко превышал десятки мегабайт и часто укладывался примерно в 100 МБ и меньше.



Нейросети сейчас

- Современные модели состоят из десятков и даже сотен слоёв, чаще всего это блоки трансформера с вниманием, нормализациями и крупными feed-forward подблоками.
- Глубина и ширина архитектуры делают модели крайне ресурсоёмкими: только хранение весов требует десятков и даже сотен гигабайт памяти.
- Общий размер весов у крупнейших современных моделей легко превышает 100 ГБ в полном формате (FP16/FP32)

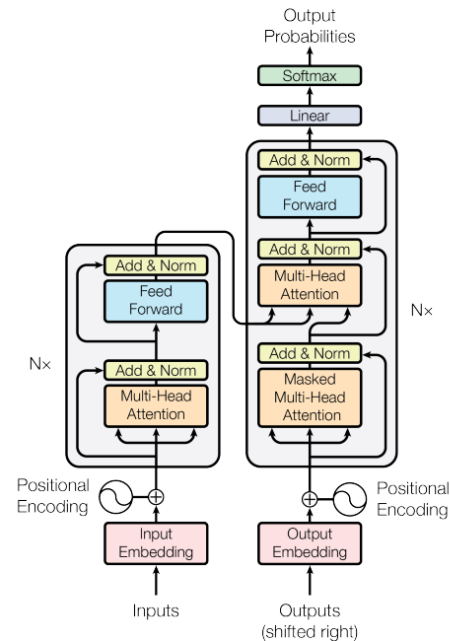


Figure 1: The Transformer - model architecture.

Как можно уменьшить размер сети?

- 1) Quantization
- 2) Pruning
- 3) Distillation

Additional topics:

- 1) Mixed precision train



Quantization

Вопросы для аудитории:



- 1) На каких девайсах обучаются современные нейросети?
- 2) Где хранятся веса модели во время обучения и инференса?
- 3) В каком формате представлены параметры (веса) нейросети?

Ответы:

1) На каких девайсах обучаются современные нейросети?

GPU/TPU серверы — видеокарты (NVIDIA A100, H100) или тензорные ускорители (TPU) с огромным объёмом памяти (80+ ГБ VRAM).

2) Где хранятся веса модели во время обучения и инференса?

Во время обучения: веса хранятся в GPU/TPU памяти (максимально быстро для градиентных обновлений). Во время инференса: веса могут быть в GPU, CPU памяти или даже на диске (медленнее, но экономнее).

3) В каком формате представлены параметры (веса) нейросети?

Формат хранения: обычно FP32 (4 байта на вес)

Разные форматы для чисел с плавающей запятой



Sign (Бит знака)

- 0 — положительное число
- 1 — отрицательное число

Exponent (Экспонента)

Определяет порядок числа, то есть сдвиг запятой. Но хранится со сдвигом (bias).

FP32: bias = 127

FP16: bias = 15

Mantissa (Мантисса)

Это дробная часть, определяющая точность числа. Кроме Int 4 и int 8.

■ Sign ■ Exponent ■ Mantissa



Пример: Получение из 2-ого (FP32) в 10-ое

$$(-1)^S * 1, M * 2^{(E-127)}$$

0 10000001 010000000000000000000000

Поле	Значение	Интерпретация
Sign	0	+
Exponent	$10000001_2 = 129$	$129 - 127 = 2$
Mantissa	$1.010000..._2$	$= 1.25$

$$(-1)^{sign} \times mantissa \times 2^{(exponent-bias)} = +1.25 \times 2^2 = 5.0$$

С каких сторон можно смотреть на квантизацию



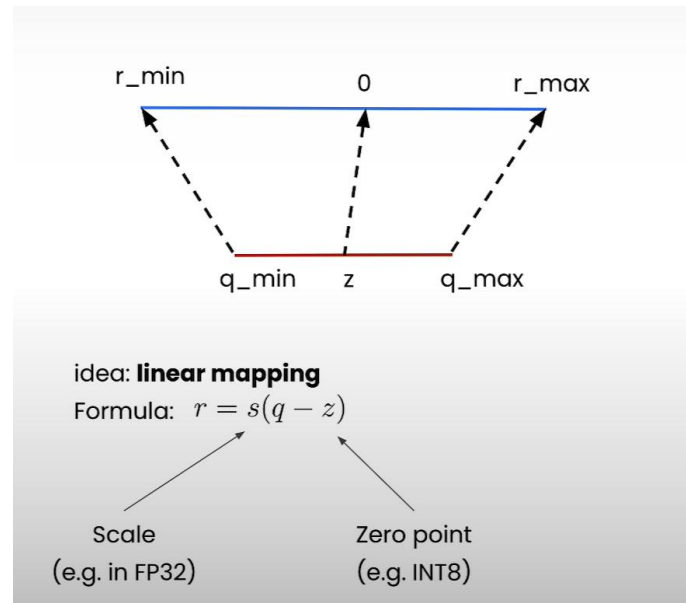
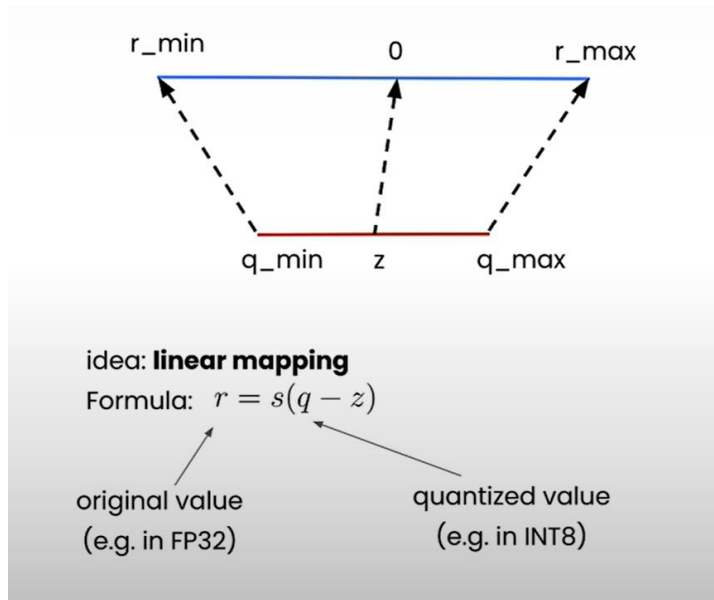
1. По объекту квантования (что квантуем)
 1. Weights quantization
 2. Activations quantization
2. По способу (методу) квантования
 1. Linear Quantization
 2. Non-Uniform Quantization
3. По моменту применения (в контексте обучения)
 1. Post-Training Quantization
 2. Quantization-Aware Training

С каких сторон можно смотреть на квантизацию



1. По объекту квантования (что квантуем)
 1. Weights quantization
 2. Activations quantization
2. По способу (методу) квантования
 1. **Linear Quantization**
 2. **Non-Uniform Quantization**
3. По моменту применения (в контексте обучения)
 1. Post-Training Quantization
 2. Quantization-Aware Training

Linear Quantization (From INT8 -> FP32)



Как из этого получить FP32 -> INT8
(Или как выразить q через r)

Linear Quantization (From FP32 -> INT8)

$$r = s(q - z)$$

$$\frac{r}{s} = q - z$$

$$\frac{r}{s} + z = q$$

$$q = \text{round}\left(\frac{r}{s} + z\right)$$

$$q = \text{int}\left(\text{round}\left(\frac{r}{s} + z\right)\right)$$

Linear Quantization (From FP32 -> INT8)

$$r = s(q - z)$$

$$\frac{r}{s} = q - z$$

$$\frac{r}{s} + z = q$$

$$q = \text{round}\left(\frac{r}{s} + z\right)$$

$$q = \text{int}\left(\text{round}\left(\frac{r}{s} + z\right)\right)$$

Что мы поняли

- 1) Как из INT8 получить FP32
- 2) Как из FP32 получить INT8

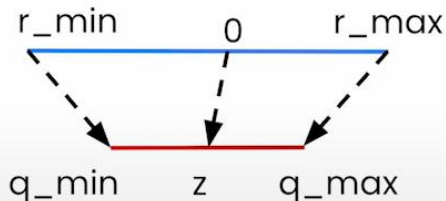
3) Что такое z и s ?

Linear Quantization. Получение s и z

Linear quantization maps the floating point range $[r_{\min}, r_{\max}]$ to the quantized range $[q_{\min}, q_{\max}]$

If we look that the **extreme values**, we should get:

$$\begin{cases} r_{\min} = s(q_{\min} - z) \\ r_{\max} = s(q_{\max} - z) \end{cases}$$



If we subtract the first equation from the second one, we get the **scale s** :

$$s = (r_{\max} - r_{\min}) / (q_{\max} - q_{\min})$$

For the **zero point z**, we need to round the value since it is a n-bit integer:

$$z = \text{int}(\text{round}(q_{\min} - r_{\min}/s))$$

s (scale) — коэффициент масштаба

z (zero_point) — сдвиг для центровки нуля

Linear Quantization. Пример

191.6	-13.5	728.6
92.14	295.5	-184
0	684.6	245.5

$$\begin{cases} [r_{min}, r_{max}] = [-184, 728.6] \\ [q_{min}, q_{max}] = [-128, 127] \end{cases}$$

$$\begin{aligned} s &= (728.6 - (-184)) / (127 - (-128)) \\ &= 3.58 \end{aligned}$$

$$\begin{aligned} z &= \text{int}(\text{round}((-128) - (-184)/3.58)) \\ &= -77 \end{aligned}$$

Linear Quantization: Symmetric vs. Asymmetric Mode



- 1) Symmetric Mode: $z = 0$
- 2) Asymmetric Mode: $z \neq 0$

Linear Quantization: Как ее можно применять

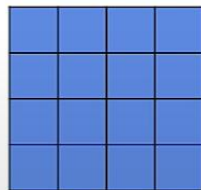
1) На всю сеть целиком:

- Вычисляем $q_{\max}$ и $q_{\min}$ по всем весам
- Вычисляем z и s

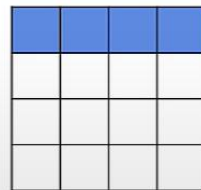
2) На кастомные блоки:

- На каждую матрицу весов
- На каждый канал/серию каналов
- На группы элементов
- Для каждого такого разделение нужно высчитывать $q_{\max}$, $q_{\min}$, z , s

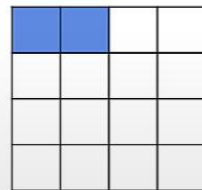
Per
tensor



Per channel
(along an axis)



Per group
(group n elements
together)



С каких сторон можно смотреть на квантизацию



1. По объекту квантования (что квантуем)
 1. **Weights quantization**
 2. **Activations quantization**
2. По способу (методу) квантования
 1. Linear Quantization
 2. Non-Uniform Quantization
3. По моменту применения (в контексте обучения)
 1. Post-Training Quantization
 2. Quantization-Aware Training

Weights & Activations Quantization



В нн можно модель квантизовать не только по **весам**, но еще и по **активациям**

От этого будет зависеть что храниться в памяти как идут вычисления

Storage	Quantized Weight + Activation (e.g. W8A32)	Quantized Weight + Quantized Activation (e.g. W8A8)
Computation	Floating point arithmetics (FP32,FP16,BF16...)	Integer based arithmetics (INT8, INT4 ...)

Note that we need to **dequantize** the weights to perform the floating point computation !

Not supported by all hardware !

Weights & Activations Quantization: Пример



```
def quantized_linear_W8A32_without_bias(input, q_w, s_w, z_w):  
    assert input.dtype == torch.float32  
    assert q_w.dtype == torch.int8  
  
    dequantized_weight = q_w.to(torch.float32) * s_w + z_w  
    output = torch.nn.functional.linear(input, dequantized_weight)  
    return output
```

Input – результат от предыдущего слоя (L-1) в FP32

Q_w – веса текущего слоя (L) в INT8

S_w, Z_w – для этих весов

output – результат работы этого слоя в FP32

Activations Quantization:



Квантизация весов работает точно также как и весами, однако есть важное отличие

Мы не знаем $q_{\max}$ и $q_{\min}$ (Вопрос аудитории почему?)

Activations Quantization :



Квантизация весов работает точно также как и весами, однако есть важное отличие

Мы не знаем $q_{\max}$ и $q_{\min}$

И чтобы их найти нужно пройти калибровку. Прогнать через оригинальную модель несколько показательных примеров и зафиксировать на каждом слое $q_{\max}$ и $q_{\min}$

Когда это произойдет → Можно будет найти s и z

Промежуточный итог



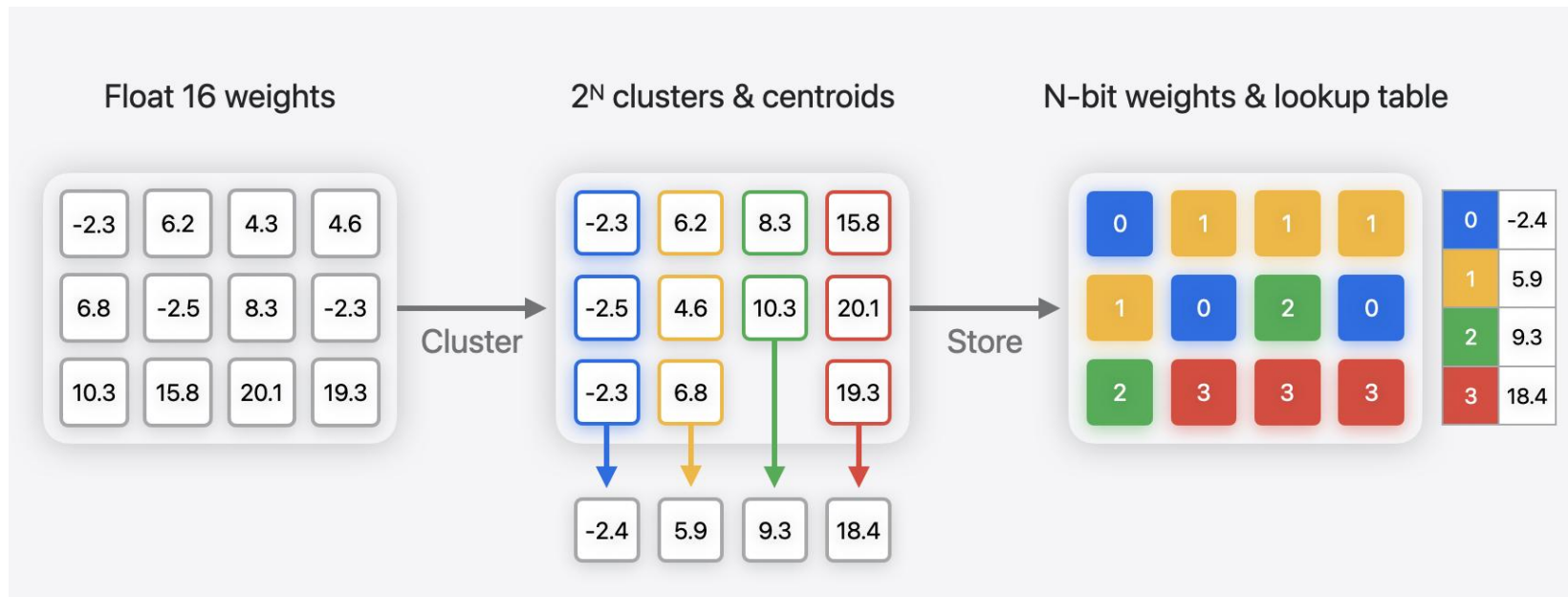
Метод	Описание инференса	Что храним дополнительно?
Только веса	при инференсе веса деквантируются обратно в float: $W_{float} = (W_{int8} - zp_W) \times scale_W$ дальше всё вычисляется в обычной float32 арифметике.	W_int8 Z S
Веса + вычисления (runtime int8)	INT8×INT8→INT32 Тут нужен специальный вид обучения QAT (разберем далее)	W_int8 Z S Z_Y S_Y
Веса + вычисления	Умножение на матрицу в INT8 Активации в FP32	W_int8 Z S Z_Y S_Y

С каких сторон можно смотреть на квантизацию



1. По объекту квантования (что квантуем)
 1. Weights quantization
 2. Activations quantization
2. По способу (методу) квантования
 1. **Linear Quantization**
 2. **Non-Uniform Quantization**
3. По моменту применения (в контексте обучения)
 1. Post-Training Quantization
 2. Quantization-Aware Training

Non-Uniform Quantization: Palettization



Linear Quantization: Outliers



С каких сторон можно смотреть на квантизацию



1. По объекту квантования (что квантуем)
 1. Weights quantization
 2. Activations quantization
2. По способу (методу) квантования
 1. Linear Quantization
 2. Non-Uniform Quantization
3. По моменту применения (в контексте обучения)
 1. Post-Training Quantization
 2. Quantization-Aware Training

Post-Training Quantization (PTQ)

Квантизация модели **после обучения** без retrain'а. Быстро, просто, но возможна потеря точности.

Все подходы, которые мы обзревали ранее это PTQ

Quantization-Aware Training (QAT)

Модель **учится с учётом квантизации**. Fake-quant в тренировке → минимальная потеря точности на int8.

Quantization-Aware Training (QAT)



Модель обучается с имитацией квантования (*fake quant*) в каждом forward-pass.

Forward (fake quantization):

$$q = \text{round} \left(\frac{x}{\text{scale}} + zp \right)$$

$$x_{\text{fake}} = (q - zp) \cdot \text{scale}$$

Backward (STE):

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial x_{\text{fake}}}$$

После обучения

$$W_{\text{int8}} = \text{round} \left(\frac{W_{\text{fp32}}}{\text{scale}_W} + zp_W \right)$$

AWQ & GPTQ & GGUF



AWQ (Activation-Aware Weight Quantization)

- Квантует веса с учётом самых «важных» активаций, сохраняя outliers в FP16.
- Минимизирует ошибку именно в тех местах, где слой чувствителен.
- Даёт стабильное качество в 4 битах и лёгкий рантайм на GPU.

GPTQ (Gradient Post-Training Quantization)

- Оптимизирует квантование весов, используя приближение Гессiana для минимизации влияния на лосс.
- Работает построчно с error-feedback, поэтому держит почти FP16-точность.
- Лучший PTQ-метод для 4 бит на больших языковых моделях.

GGUF (формат хранения, НЕ метод квантования)

- Это упаковка весов в удобный бинарный контейнер для CPU-ориентированного inference.
- Поддерживает разные типы квантования (Q2–Q6, FP16, INT8), но сам квант не выполняет.
- Максимально портируемый формат для llama.cpp, Ollama и мобильных устройств.

Pruning

Pruning (прунинг) — это метод оптимизации нейросетей, при котором из модели удаляются наименее значимые веса, нейроны или целые структуры (каналы, головы внимания, блоки).

Цель — уменьшить размер модели и ускорить инференс без критической потери качества.

Прунинг может быть неструктурным (удаление отдельных весов), структурным (удаление целых компонент) или блоковым (паттерны N:M), применяться после обучения или итеративно с последующим fine-tuning.

Magnitude Pruning (L1/L2-based)



Идея: если модуль веса маленький $\rightarrow$ вклад в выход минимальный $\rightarrow$ можно удалить.

Как работает:

- 1) считаем $|w|$ или w^2 для каждого веса,
- 2) сортируем
- 3) обнуляем низкий процент (например, 20%).

Плюсы: быстро, легко

Минусы: не учитывает контекст лосса и активаций.

Weight Thresholding (Static Pruning)



вводится фиксированный порог τ : если $|w| < \tau \rightarrow$ обнулить.

Используется как baseline и в ранней научной литературе.

Хорошо подходит для toy-примеров и предварительного анализа чувствительности.

Iterative Magnitude Pruning (IMP)



Алгоритм:

- 1) режем по magnitude 10–20% весов,
- 2) делаем fine-tuning,
- 3) повторяем несколько раз пока качество значительно не снизится

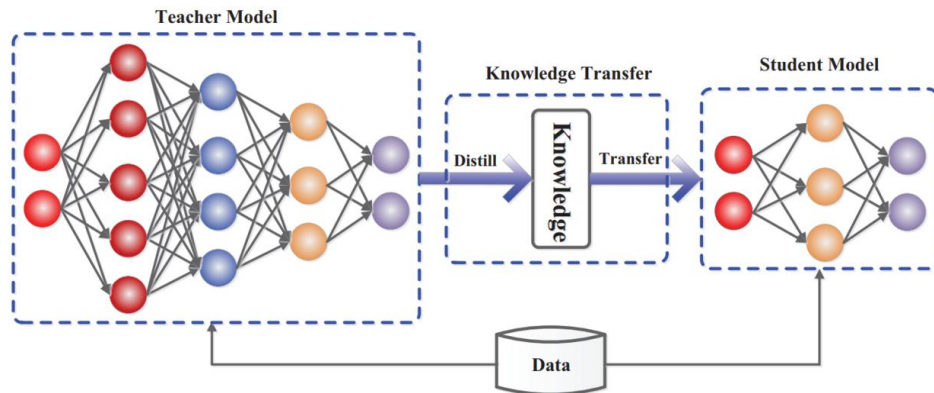
Даёт существенно лучшее качество, потому что модель адаптируется.

Knowledge Distillation

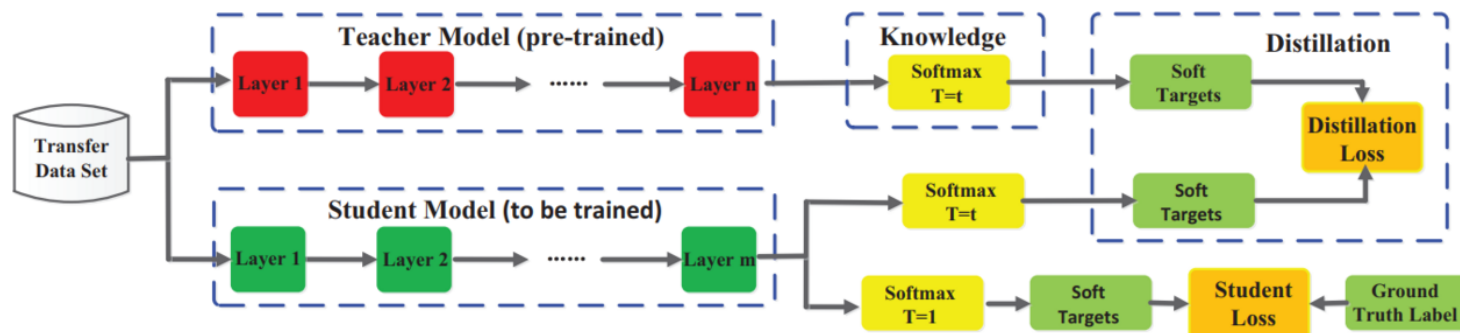
Knowledge Distillation

Knowledge Distillation — это метод, при котором большая, хорошо обученная модель (*teacher*) передаёт своё поведение более компактной модели (*student*).

Студент учится не только на истинных метках, но и на «мягких» распределениях вероятностей, которые выдаёт teacher, тем самым перенимая его знания и структуру решений.



Knowledge Distillation: classic method

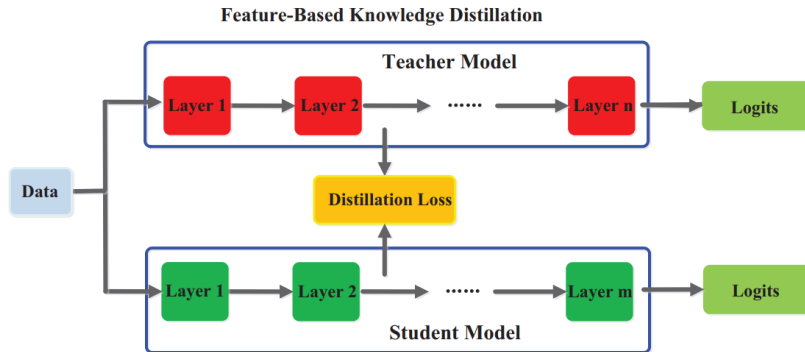


$$\mathcal{L} = \underbrace{(1 - \alpha) \text{CE}(y, \text{softmax}(z_s))}_{\text{hard targets}} + \underbrace{\alpha T^2 \text{CE}\left(\text{softmax}\left(\frac{z_t}{T}\right), \text{softmax}\left(\frac{z_s}{T}\right)\right)}_{\text{distillation loss (soft targets)}}$$

Knowledge Distillation: other methods

Дистилляция на основе признаков (feature-based distillation)

Ученик учится не только по финальному выходу модели, но и по промежуточным представлениям учителя — картам признаков и активациям скрытых слоёв.

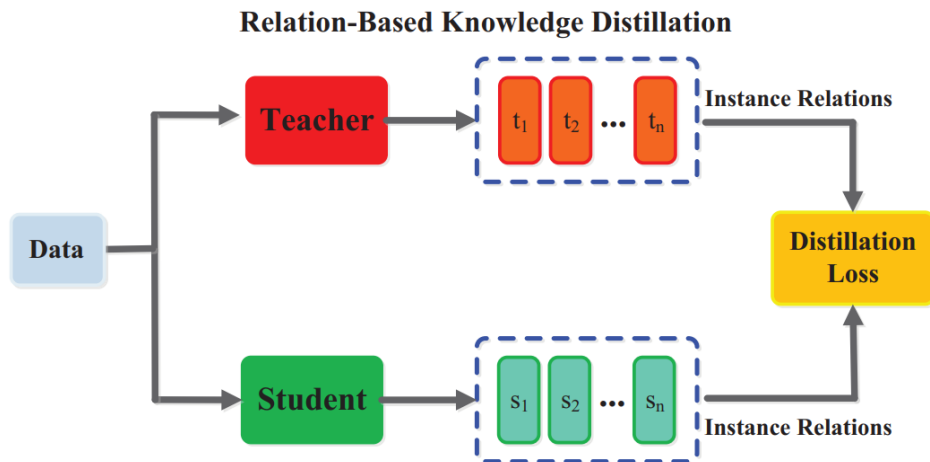


Этот подход особенно повышает качество в задачах компьютерного зрения, где структура признаков играет критическую роль.

Knowledge Distillation: other methods

Дистилляция на основе отношений (relation-based distillation)

Ученик обучается воспроизводить отношения, которые учитель формирует между разными слоями или разными выборками данных.



Такой подход может объединять знания нескольких учителей и подходит для более сложных, многомодельных сценариев.

Knowledge Distillation: classification



Офлайн-дистилляция — сначала обучается преподаватель, а затем он обучает студента.

Онлайн-дистилляция (пример SimSiam) — преподаватель и студент учатся вместе.

Самодистилляция (пример DINO) — студент учится самостоятельно.

Knowledge Distillation: classification



Тип дистилляции	Модель-учитель	Подход к обучению	Преимущества	Сложности
Оффлайн-дистилляция	Предобученная и фиксированная	Сначала обучается учитель, затем знания передаются студенту	Простая, эффективная, учителя можно многократно использовать	Студенту может быть сложно достичь уровня учителя из-за <i>разрыва в знаниях</i>
Онлайн-дистилляция	Обучается вместе со студентом	Учитель и студент учатся одновременно в реальном времени	Адаптивное обучение, <i>нет необходимости в заранее обученном учителе</i>	Вычислительно дорогой и более сложный процесс
Самодистилляция	Отдельная модель-учитель не требуется	Модель обучает сама себя, используя свои более глубокие уровни	Не нужен учитель, повышает эффективность	Может уступать по качеству обучению от сильной внешней модели-учителя

Knowledge Distillation: advanced methods



Multi-teacher KD: объединение знаний нескольких учителей → более стабильный и разносторонний студент.

Attention KD: студент перенимает карты внимания учителя — учится «куда смотреть».

NTCE-KD: дистилляция вероятностей всех классов, включая ошибочные и слабые.

Adversarial / Data-free KD: GAN генерирует синтетические данные или помогает сравнивать поведение учителя и студента.

Quantized KD: обучение студента сразу в низкой разрядности (8-bit, 2-bit).

KD for Generative Models: ускорение диффузионных и autoregressive моделей за счёт упрощённого студента.

NAS-driven KD: автоматический подбор оптимальной архитектуры студента.

Mixed precision training

Вопросы для аудитории:



Студент Василий, первокурсник, поставил тренироваться свою нейронную сеть с оптимизатором **SGD** и ушёл домой.

Утром он обнаружил, что сеть так и не дообучилась. Тогда он обратился к своему товарищу Пете за советом, и тот порекомендовал попробовать оптимизатор **Adam** — мол, он «круче».

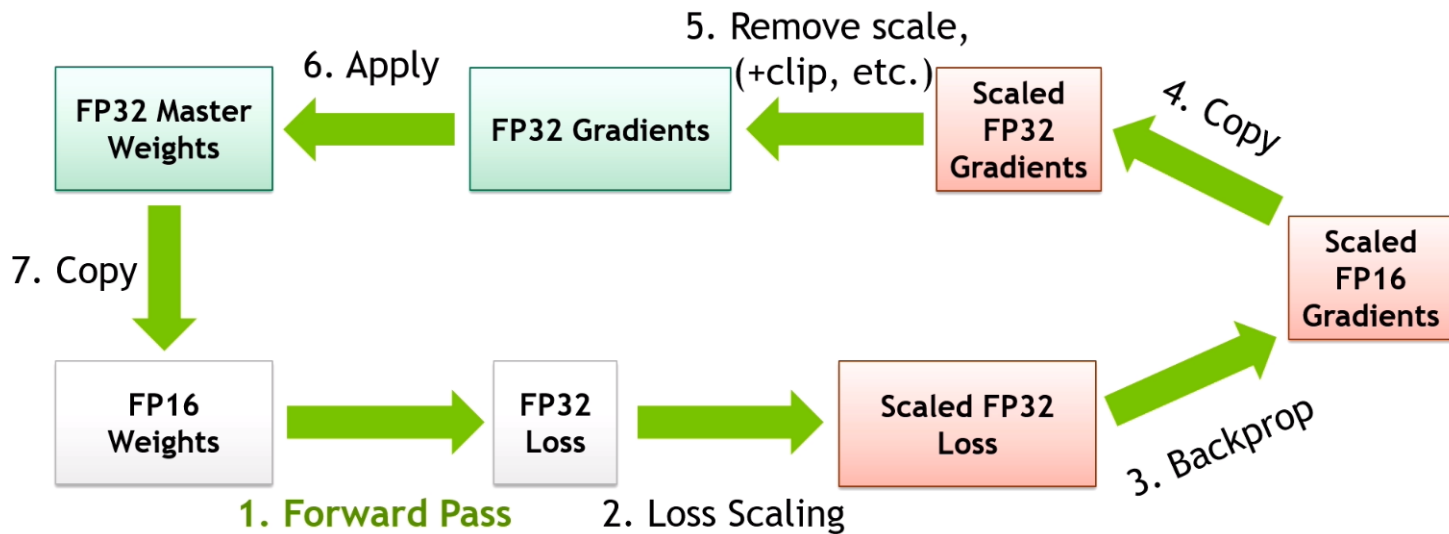
Василий запустил тот же эксперимент, но заменил SGD на Adam — и получил ошибку ***Out of Memory***.

Почему так произошло?

- 1) SGD хранит 2 копии параметров: сами веса и их градиенты.
- 2) Adam хранит 4 копии параметров: веса, градиенты, первый момент и второй момент.
- 3) Это означает, что Adam требует примерно в 2 раза больше GPU-памяти, чем SGD.
- 4) Если модель уже была близка к лимиту памяти, переход на Adam приводит к Out of Memory.

Mixed precision

MIXED PRECISION TRAINING



Mixed precision

1) FP16 Weights → Forward Pass

- Модель считает forward в FP16 (быстрее, меньше памяти).
- Функция потерь считается в FP32 для стабильности.

2) Loss Scaling

- FP32-loss умножается на scale-factor,
- чтобы предотвратить исчезающие FP16-градиенты.

3) Backward в FP16

- Backprop вычисляет scaled FP16-градиенты
- (они увеличены из-за scaling).

4) Copy FP16 → FP32

- Scaled FP16-градиенты конвертируются в FP32 —
- так безопаснее выполнять деление и операции оптимизатора.

5) Unscale Gradients

- FP32-градиенты делятся на scale-factor → получают нормальные значения.
- На этом этапе можно делать clip и проверку overflow.

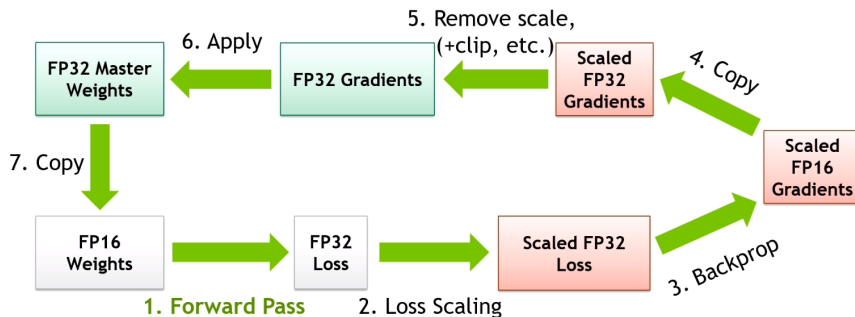
6) Обновление FP32 Master Weights

- Оптимизатор обновляет только FP32-веса —
- они служат «истинной» и точной версией модели.

7) Copy FP32 → FP16

- Обновлённые FP32-веса конвертируются обратно в FP16 —
- чтобы следующий forward/backward снова выполнялся быстро.

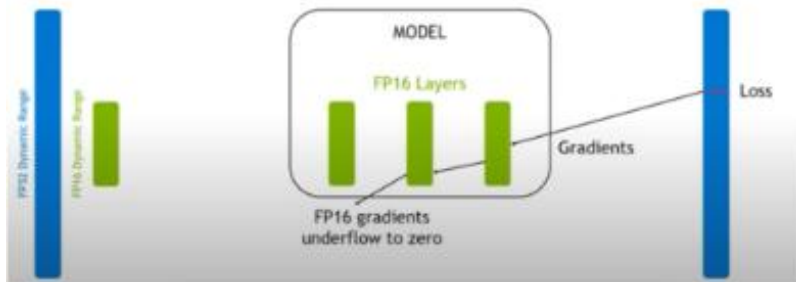
MIXED PRECISION TRAINING



Для чего нужен scaling?

GRADIENT UNDERFLOW

Small gradients may underflow in FP16 regions of the network.



LOSS SCALING

Scaling the loss brings gradients into the FP16 dynamic range.



Mixed precision



ТИП ОБУЧЕНИЯ	СКОРОСТЬ РАБОТЫ	Кол-во параметров для обучения (SGD)
Full Precision	-	2 * Model size
Mixed precision	×1.5–2.5 ускорение	? * Model size

Mixed precision



ТИП ОБУЧЕНИЯ	СКОРОСТЬ РАБОТЫ	Кол-во параметров для обучения (SGD)
Full Precision	-	2 * Model size
Mixed precision	×1.5–2.5 ускорение	3 * Model size (FP32 model, FP32 gradients, FP16 model, FP16 gradients) Но за счёт сокращения активаций FP16 экономит ~30–50% общей VRAM и дает возможность обучать большие модели.

Mixed precision в коде

Python



```
import lightning as L

# ... define your LightningModule and DataLoader ...

trainer = L.Trainer(precision="16-mixed")
trainer.fit(model, dataloader)
```

**Спасибо
за внимание!**

it'sMO *re than a*
UNIVERSITY